AGENTTEAMS · 数字员工实践

基于 AgentTeams 的数字员工落地实践

一项职能=一支 Agent 团队：可复制的数字员工工厂

- 批量构建
- 标准交付
- 持续运营

不是单点机器人，而是方法论与工程化实践

LEADER — WORKER 协同

背景 · 为什么需要数字员工

研发组织的“职能类工作”之痛

01 · 重复消耗

大量工时被重复劳动吞噬

值班答疑、工单处理、社区 Issue、取数提单，**日复一日消耗研发工时**。

02 · 频繁打断

碎片任务打断主线工作

碎片化请求不断插入，工程师**难以持续专注**于核心研发主线。

03 · 专家单点

关键知识集中在少数人

专家成为单点瓶颈，权限只能**“全有或全无”**，无法安全分级下放。

04 · 7×24 压力

响应压力不分昼夜

社区与告警**不分时区、不分工作日**，团队被迫 7×24 待命响应。

核心破题思路

一项职能 = 一支 Agent 团队

> 我们不是在做一个**孤立的机器人**，而是把一项职能完整交给一支**可协同、可治理、可进化**的 Agent 团队。

可协同

Leader × Worker 分工，规划与执行各司其职

可治理

权限矩阵、审计追踪与安全护栏兜底

可进化

经验沉淀持续回流，越用越懂业务

从机器人到数字员工团队：范式升级

传统脚本 / 单点机器人

按指令执行的“工具”

- ✕ **固定流程：**只能按预设脚本执行，无法应变
- ✕ **单一技能：**一个机器人只会做一件事
- ✕ **无法拆解长任务：**复杂链路一断即停
- ✕ **出错难追溯：**黑盒执行，定位问题靠人工
- ✕ **经验不沉淀：**每次执行都从零开始

VS

AgentTeams 数字员工团队

推荐

主动交付结果的“团队”

- ✓ **多角色协作：**Leader 规划 + Worker 专业执行
- ✓ **任务拆解与并行：**长任务自动分解、同步推进
- ✓ **按需调用工具与技能：**动态编排，灵活应变
- ✓ **全程可追溯：**计划、执行、报告透明可审计
- ✓ **经验资产化：**成果沉淀复用，持续进化

→ **范式升级：**从「执行一个动作」的自动化，走向「交付一件完整事情」的团队式智能

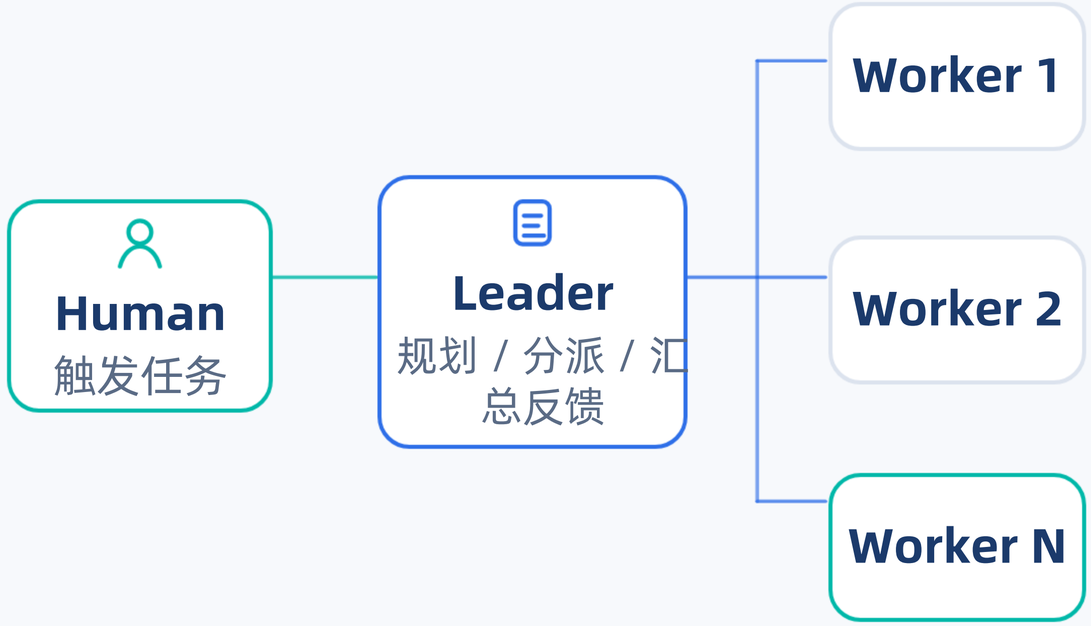
AgentTeams 协同模型：Leader × Worker

Team = Human + Leader + N Workers

Human 触发，Leader 规划、分派并汇总反馈，Worker 专业执行

Team 协同结构

Leader 和 Worker 都是 Agent



Leader × Worker 分工

Leader 负责中枢决策

理解意图、制定 Plan、选择工具与 Worker，并统一管理上下文。

Workers 专业化执行

按领域拆分子任务，分别调用知识、系统、工具完成可验证动作。

反馈归 Leader 汇总

Worker 只回传执行结果，由 Leader 汇总为报告或操作结论，再反馈给 Human。

规模化复制全景

数字员工团队矩阵全景

从样板间到 Team 与 Agent，把客户场景沉淀为可配置、可治理、可复用的数字员工工厂。

7 / 25

样板间矩阵
7 个样板间
25 个子样板间

49 / 75

Team 与 Agent
49 个 Team，承载 75 个 Agent

300+

覆盖用户
面向业务用户形成使用闭环

18+1

管理员治理
分层运营，保障复制质量



样板间是复制单元
把流程、角色与知识固化为可复用配置

Team 承载职能
多 Agent 协同，支撑端到端任务交付

治理保障复制
统一管理员体系，沉淀运营与质量标准

工程视角：样板间代码组织形式

AI Agents 团队 代码仓库 实际仓库

```
ai-agent-teams/
├── skills/ ← 仓库级公共能力池
└── 样板间2：值班答疑/
    └── observe-oncall/
        ├── skills/ ← 样板间级 Skills
        ├── observe-oncall-leader/
        │   ├── config/
        │   │   ├── SOUL.md 身份与边界
        │   │   ├── AGENTS.md 协作契约
        │   │   ├── MEMORY.md 长期记忆
        │   │   ├── BOOTSTRAP.md 启动自检
        │   │   └── HEARTBEAT.md 运行心跳
        │   ├── skills/ ← Agent 本地 Skills
        │   └── ref_skill.json ← 声明式引用清单
        ├── observe-oncall-cmsv1-agent/
        ├── observe-oncall-cmsv2-agent/
        └── observe-oncall-cloudlens-agent/
```

五件套配置规范

SOUL 身份 | AGENTS 协作 | MEMORY 记忆 | BOOTSTRAP 启动 | HEARTBEAT 心跳

配置即契约：我是谁、和谁协作、记住什么、如何稳定运行

三层 Skill 架构

仓库级 skills — 跨项目通用 | 样板间级 skills — 场景模板 | Agent 本地 skills — 角色专属

ref_skill.json 声明引用，打包时就近向上注入

先本地验证 → 沉淀样板间 → 进入仓库级共用

配置即代码 · Agent 可复制 · Skill 可复用

落地流程：从定义到进化的闭环

数字员工全生命周期六步走：交付不是终点，运行经验持续回流、持续进化

01

定义

五件套配置规范 + 技能编写，定义身份、契约与能力边界

02

规范检查

路径 / 映射 / 凭证 / Git 四项审计，发布前逐项把关

03

打包发布

统一打包 + 模版发布，两步走灰度防止打穿全量

04

批量建站

五阶段自动化 + AI 语义审核 + 断点续连，规模化交付

05

运行监控

拨测探活 + 响应时延跟踪，持续掌握运行健康度

06

回流进化

运行时变更回流 Git，经验沉淀反哺定义与技能

场景一痛点：开源社区维护

案例场景描述

Apache RocketMQ 是阿里开源的分布式消息中间件，GitHub 上持续产生大量 Issue 和 PR。核心维护团队有限，需要自动化方案保障**响应速度**与**代码质量**。

关键挑战：用自动化放大有限维护人力。

传统方式的痛点

01	**Issue 响应周期长**	人工确认、定位、修复方案平均 3-5 个工作日
02	**Code Review 瓶颈**	单个 PR 需阅读 diff、上下文和安全/兼容性，耗时 30-60 分钟
03	**跨仓库一致性难保证**	rocketmq 与 rocketmq-clients 协议同步易遗漏
04	**新贡献者 CI 审批延迟**	首次贡献需 maintainer 手动 approve，受时区影响
05	**社区活跃度受影响**	响应慢导致贡献热情下降，Issue 堆积



开源社区维护数字员工团队

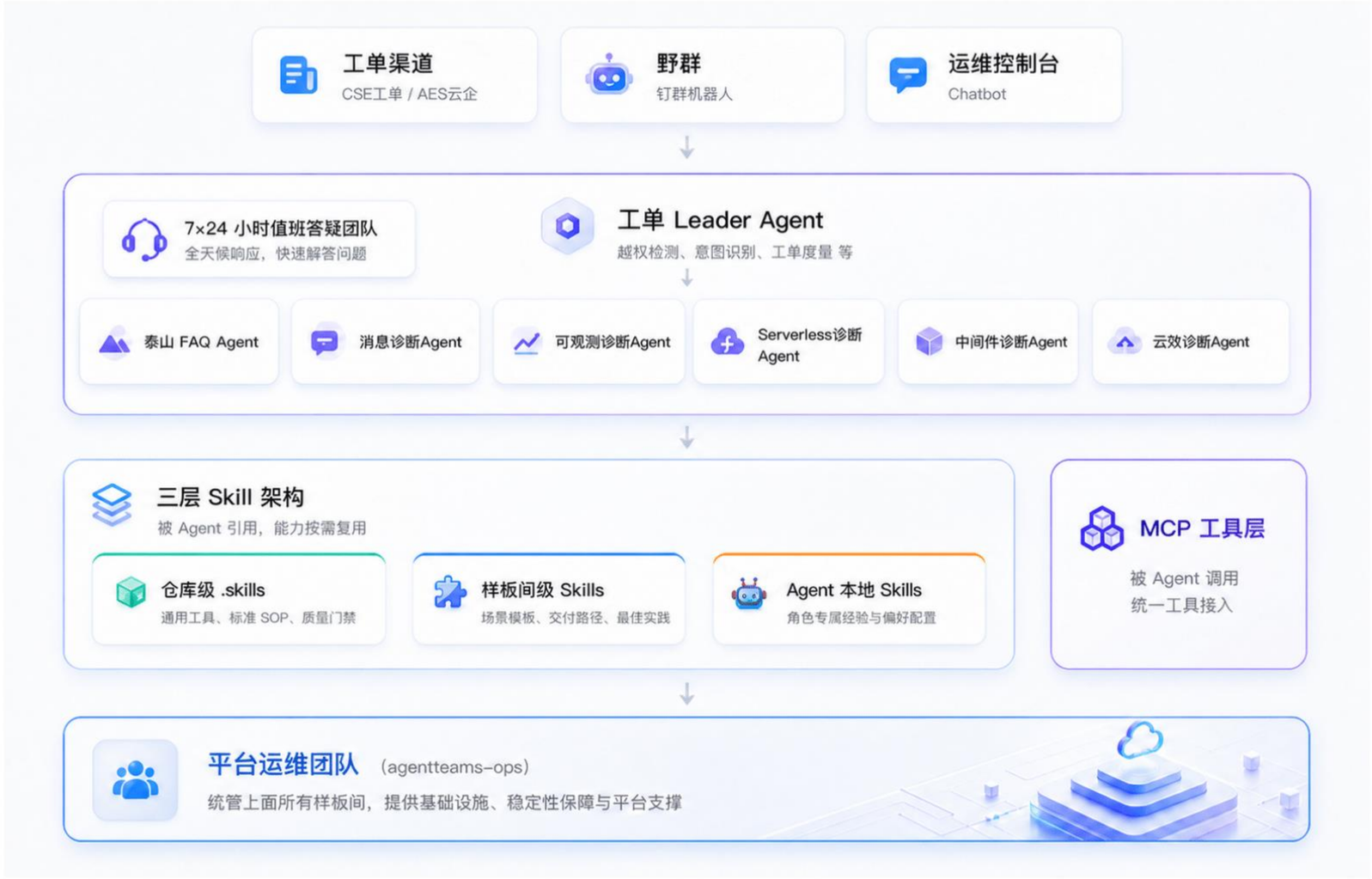
全链路闭环：
- issue → Spec
- 人工：{/approve}
- PR Review→Merge

效率提升
首响 3-5天→1小时内。

质量改善
Review 自动且覆盖率 100%。

成本优化
节省约 60% 维护时间。

创新价值
覆盖 10+ 个社区 / 25+ 个仓库。



覆盖 9 个产品、22 个诊断 Agent、86 个诊断业务技能

场景二：Oncall 值班与专业分诊

01 多入口接入
工单、群聊、控制台统一进线。

02 Leader 分诊
识别意图并派发专业 Agent。

03 专业诊断
FAQ、可观测、云产品协同诊断。

04 能力复用
Skill 与 MCP 工具层稳定答疑。

协同结果
专家能力编排可复用，可度量

工单度量：工单 AI 覆盖金字塔

闭环率和工单逃逸率看 Agent 诊断能力；**场景逃逸率**看能力覆盖缺口。

AI 闭环率

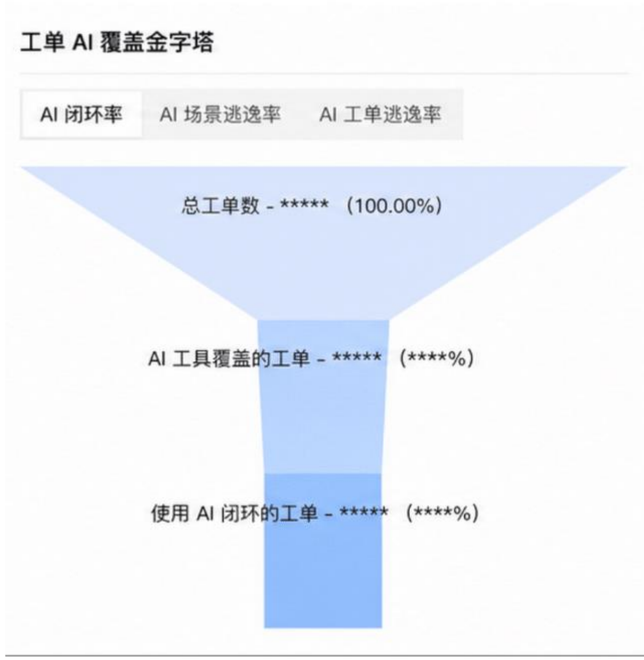
场景已覆盖时，诊断 Agent 无需人工升级即可完成闭环的能力。

AI 工单逃逸率

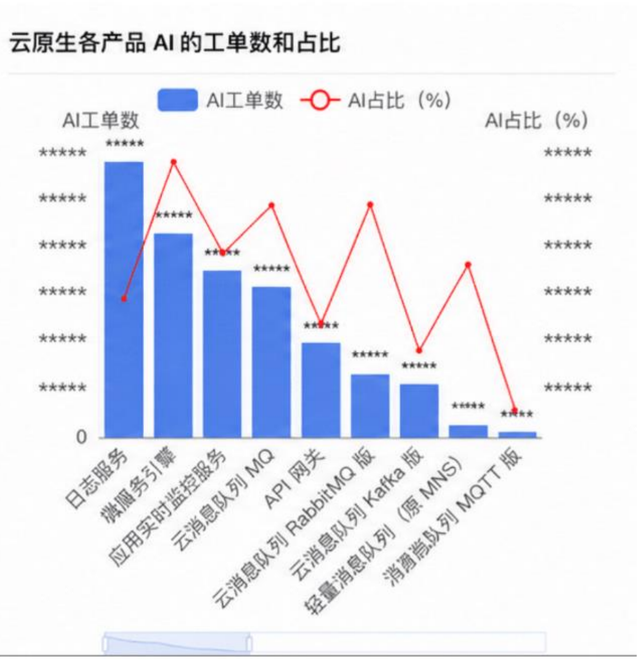
场景已覆盖且已调用 AI，但仍需升级人工，反映诊断 Agent 能力短板。

AI 场景逃逸率

诊断场景尚未覆盖，需补充知识库、诊断工具或专项能力。







场景三：应急数字人协同处置

一切围绕缩短 **MTTR**：应急指挥 Leader 统筹调度，四个 Worker 并行协作、闭环处置

衡量指标：MTTA/MTTR 改善 / 根因定位命中率 / 恢复验证覆盖率

应急指挥 Leader

接警分类 · 任务分派 · 全程跟踪闭环

响应红线

- 5分钟内分类分派
- P0/P1 即时通知
- 超时自动升级

01

信息同步

统一事件口径，实时播报进展，拉通全部干系人

02

根因分析

聚合日志与指标，快速收敛疑点，定位故障根因

03

恢复处置

执行预案与回滚，优先恢复业务，动作全程留痕

04

恢复验证

核验指标回归基线，确认业务恢复，闭环关闭事件

MTTR 链路 故障发现 → 分类分派 → 并行协作 → 恢复验证 → 事件关闭

场景四：经营分析团队与自然语言问数

自然语言问数 01

业务同学直接提问收入、客户、目标达成类问题，自动生成查询语句取数，无需等排期。

示例：“本季度华东区收入目标达成情况如何？”

口径铁律 02

指标口径版本化沉淀、持续更新，全团队统一口径，避免“同名指标、各说各数”。

一个指标，只有一个权威口径

准确性和效率的平衡 03

PTC（Programmatic Tool Calling，编程式工具调用）：查询一次沉淀为脚本。产品经营日报、周报查数等。

数据源未接入就如实说明查不到，绝不编造数字；每个结论都可回溯到取数来源。

分析协作 04

问数 Leader 统一调度多个分析 Worker 并行取数与分析，共同支撑业务工作台。

1 个 Leader + N 个 Worker 协同作战 | 衡量指标：问数命中率 / 口径匹配准确率 / 取数耗时下降

你敢让 AI 做生产变更吗？

AgentTeams OPS：“使用 AI 团队 运维 AI 团队”

误操作　越权　不可追溯

LLM 是“概率性执行者”

输出天然带随机性。真实教训：一次调用**全量覆盖接口**，导致成员被**误删**。

Agent 是自我进化的“活体”

运行时自改配置，会与版本库**脱钩**，持续产生**配置漂移**。

答案，是把安全判断搬出模型——不由 AI 自己说了算。

见下一页 →

可信护栏：可解释、可追溯、可拒绝

衡量指标：权限覆盖率 / 拦截次数 / 审计完整率

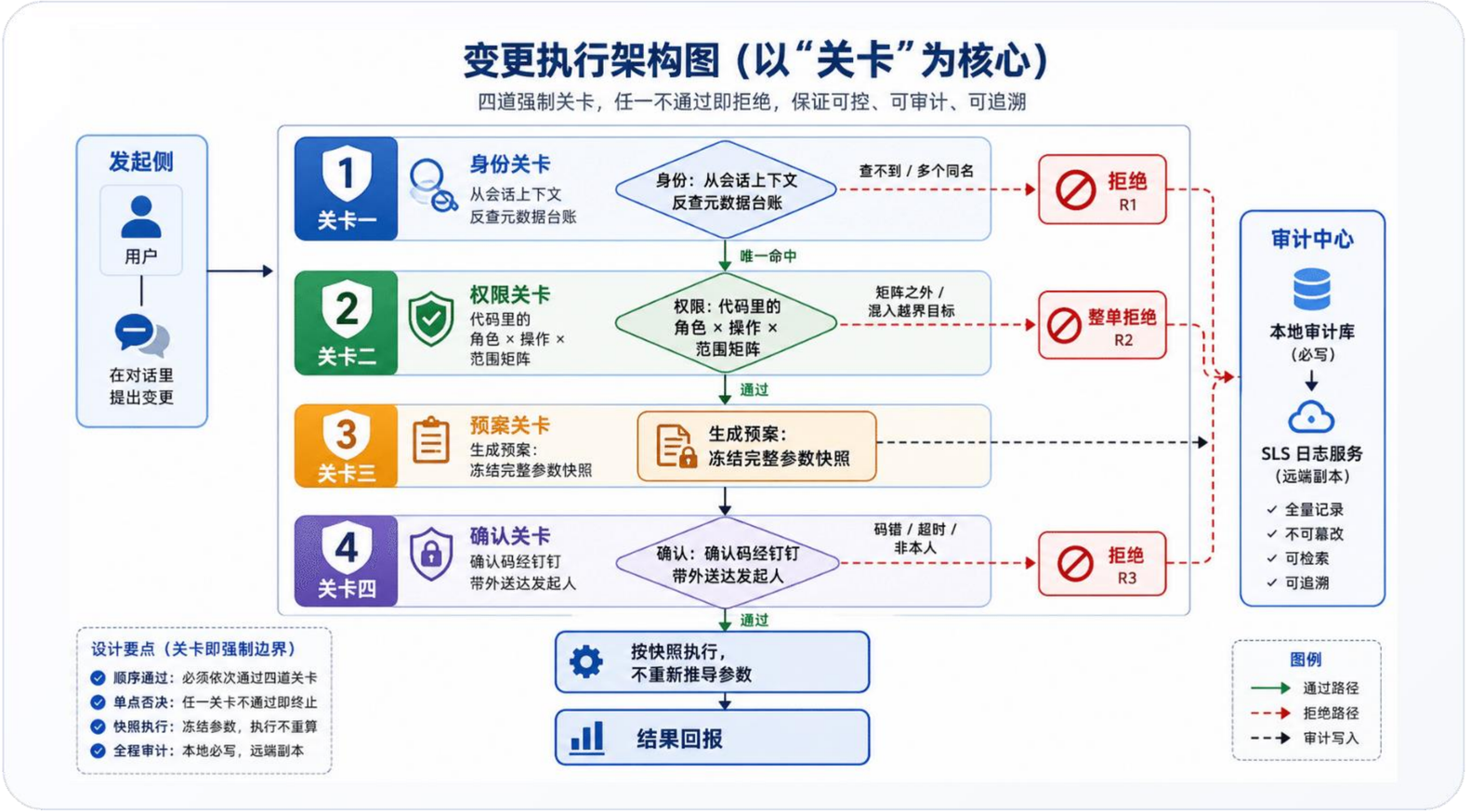
执行前门禁

把四道关卡压进外部治理链路，模型只给建议，代码决定能否执行。

- **身份**
  调用者与目标资源一致
- **权限**
  按矩阵裁决可做什么
- **预案**
  影响面与回滚先写清
- **确认**
  确认内容绑定执行内容

AgentTeams Ops 团队 四道关卡治理链路

四道关卡把高危操作纳入可裁决、可审计、可拒绝的外部治理链路。



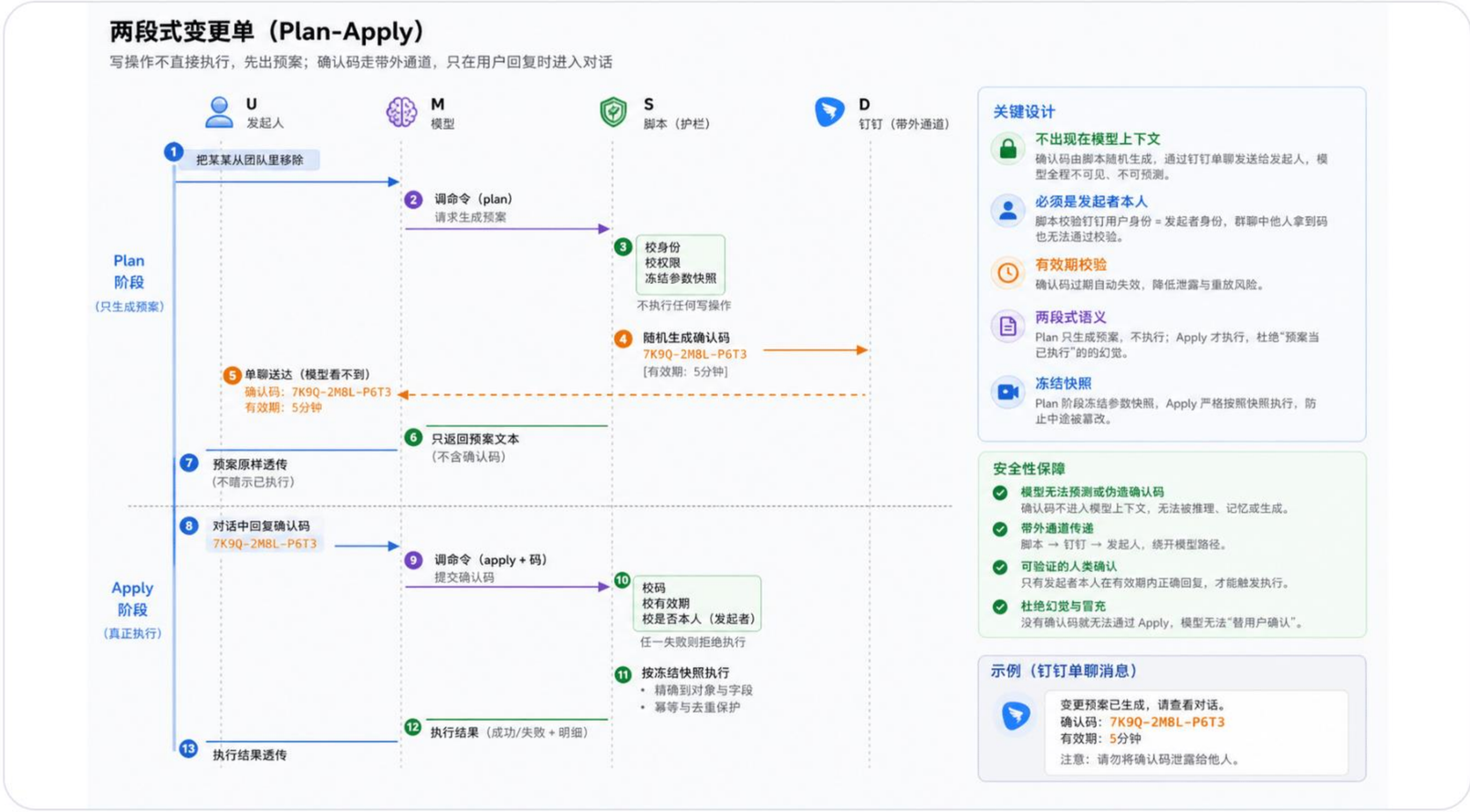
- **可解释**
  变更范围、理由可见
- **可追溯**
  全程留痕，记录SLS
- **可拒绝**
  风险拦截，快速失败

高危操作治理：两段式确认机制

协议只留三件事

1. 先冻结 **payloadhash**，锁定变更意图。
2. **带外随机码**经钉钉发送，原通道回填。
3. 通过 **本人 + TTL + hash** 校验后执行。

确认链路绑定意图、确认人、时效与 payload。

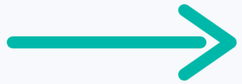


随机码，不会出现在LLM上下文中，防止模型绕过审计执行任何高危操作。

用 AI 运维 AI：从管理员瓶颈到自助运营

1 名系统管理员

人工执行·单点瓶颈



18 人安全自助运营

授权自助·审计闭环

统一管理

运维数字员工统一承接生产资源入口

分级授权

权限按角色下放，让业务团队自助完成

审计闭环

每次操作留痕，可解释、可追溯、可复核

角色升级

管理员从人工执行者升级为规则制定者

这本身就是数字员工的最佳实践 ——Dogfooding：用 AI 运维 AI

可观测与自进化：数字员工也要被运营

上线只是起点：数字员工需要持续拨测、审计回流与版本化发布，才能长期可信。



可观测

除了 Loongsuite 指标采集 + ProbeAgent 探针，保障Agent可用性。



进化回流

Badcase 与经验进入 MR、Review、Release，形成可回滚的工程闭环。

落地价值：从工具提效 到 AI Native 组织能力升级

人力释放

职能类打断性工作移交数字员工，真人回归主线

7×24 在线

黑灯工厂、值班答疑、github社区、应急全天候第一时间响应

规模化复制

样板间模式，新团队接入边际成本递减

中心主张



数字员工融入日常协作，能力沉淀为组织资产

可信可控

权限矩阵、两段式确认、全量审计，零越权零误删

知识资产化

口径、SOP、经验版本化沉淀，人员流动不断层

规模化验证

7 样板间　　**49** 接入团队

75 数字员工　　**1→18** 问题自助

THANKS

欢迎交流 / Q&A